

When Backtests Guess: How Trading Platforms Silently Fabricate Results

Michael Claeys
Independent Researcher
February 2026

SSRN Working Paper

Abstract

Most algorithmic trading platforms test strategies using OHLC (Open, High, Low, Close) bar data, which throws away all information about how price moved within each bar. When a trade's stop loss and take profit both fall inside a single bar's range, the platform has no way to know which one was hit first. It has to guess. This paper defines this problem formally, then measures how often it actually happens. Analyzing 2,064,460 one-minute bars of E-mini NASDAQ-100 futures data from 2020 through 2025, I find that for a typical scalping setup with a 10-point stop and 10-point target, 18.47% of all bars are ambiguous. The gap between best-case and worst-case assumptions reaches 3,695 NQ points (\$73,900) per 1,000 trades. I compare how five major platforms handle this ambiguity and find that each one uses a different method, none of which is provably correct without tick data. The paper provides practical guidelines for when OHLC backtesting can be trusted and when it cannot.

Keywords: backtesting, algorithmic trading, OHLC, fill models, scalping, market microstructure, phantom fills, E-mini NASDAQ-100

JEL Classification: G11, G14, C63

1. Introduction

Over the past decade, algorithmic trading tools have become widely accessible to retail traders. Platforms like QuantConnect, TradingView, Backtrader, NinjaTrader, and MetaTrader 5 serve millions of users who build and test automated strategies. All of these platforms share a common foundation: they store historical price data as OHLC bars (Open, High, Low, Close) at resolutions ranging from one minute to daily.

OHLC is a natural way to compress price data. Each bar summarizes its time interval into four numbers: the first price, the highest, the lowest, and the last. This is efficient. A full day of one-minute NQ futures bars is about 1,380 data points, compared to hundreds of thousands of individual ticks. But this compression throws away something important: the order in which the High and Low occurred. This paper shows that this missing information has real consequences for strategy evaluation, especially for scalping strategies with tight stops and targets.

Here is the core problem. A trader enters a long position on NQ with a 10-point stop loss and a 10-point take profit. On the next one-minute bar, the High is 15 points above entry and the Low is 12 points below. Both the stop and the target fall within the bar's range. Did price rise to the target first, locking in a win? Or did it drop to the stop first, taking a loss? From OHLC data alone, there is no way to tell. The platform has to pick one, and that choice directly determines whether the trade is a +10 or a -10.

I call this the **intra-bar ambiguity problem**. Practitioners have recognized this informally for years. Forum threads on every major platform discuss stop-loss and take-profit orders both filling on the same bar. But no prior work has systematically measured how often this happens, how much it distorts performance numbers, or how each platform deals with it.

This paper does four things. First, it gives a formal definition of intra-bar ambiguity. Second, it measures ambiguity frequency across multiple stop/target configurations using over 2 million one-

minute NQ bars spanning six years. Third, it documents and compares how five major platforms resolve this ambiguity. Fourth, it proposes practical guidelines for when you can trust an OHLC backtest and when you cannot.

2. Related Work

2.1 Backtesting Pitfalls

Academic work on backtesting problems has mostly focused on statistical issues. Bailey, Borwein, López de Prado, and Zhu (2014) showed that the probability of finding a spuriously profitable strategy grows rapidly with the number of configurations tested. Harvey and Liu (2015) addressed multiple testing in factor discovery. White (2000) introduced the Reality Check bootstrap for evaluating strategy performance against benchmarks. These contributions ask whether a strategy's track record is statistically meaningful, but they assume the performance itself is measured correctly. This paper challenges that assumption.

2.2 Execution Simulation

Professional quant firms simulate execution using tick-level or order-book data, modeling market impact, queue priority, and latency (Almgren & Chriss, 2001; Cont, Stoikov, & Talreja, 2010). The gap between institutional and retail simulation quality is large. Institutional simulators model whether a limit order actually gets filled based on queue position. Retail platforms fill any limit order whose price falls within the bar's range. But the specific issue of not knowing the intra-bar price path has gotten almost no formal attention.

2.3 The Gap

Existing work covers what to test (avoid overfitting, control for multiple comparisons) and what to simulate (slippage, commissions, market impact). The question of how accurately the simulation engine resolves each fill at the bar level has been mostly ignored. This paper addresses that gap:

not whether a strategy's signal has predictive power, but whether the backtest can even determine the outcome of each trade given the data available.

3. The Intra-Bar Ambiguity Problem

3.1 Definition

Take a trade with entry price P^e , stop loss at P^s , and take profit at P^t . For a long position, $P^s < P^e < P^t$. Given a bar with OHLC values $\{O, H, L, C\}$, I define an **ambiguous bar** as any bar where $L \leq P^s$ AND $H \geq P^t$. Both exit prices fall within the bar's range, and since OHLC data does not encode the order prices were hit, the trade outcome cannot be determined.

3.2 Simplified Measurement

For empirical measurement, I simplify this. Rather than tracking specific entries, I count bars whose range ($H - L$) exceeds a given stop/target width. If the bar's range covers both the stop and target distance, that bar is potentially ambiguous for any trade entered within it. This gives an upper bound on ambiguity frequency that does not depend on any specific strategy's entry logic.

3.3 Why This Creates Bias

This is not just noise. It is systematic bias. When a platform resolves an ambiguous bar, it applies a deterministic rule. If that rule consistently favors one outcome (say, always assumes the take profit is hit first), the bias compounds over hundreds or thousands of trades. For strategies with asymmetric risk/reward (10-point stop, 20-point target), the platform's guess on an ambiguous bar determines whether a trade is recorded as +20 or -10. That is a 30-point swing on a single trade, decided by a coin flip dressed up as a simulation.

4. How Each Platform Handles It

I reviewed the documentation, source code, and community forums of five major retail platforms. The variation is striking: no two use the same approach.

4.1 QuantConnect (LEAN Engine)

QuantConnect's open-source LEAN engine processes pending orders against each bar sequentially. When both a stop market order and a limit order fall within one bar's range, the engine fills both on the same bar. The trader ends up holding a reversed position instead of being flat. This cannot happen with a real broker using bracket orders. The official recommendation is to increase data resolution or widen stops. For stop fills specifically, LEAN's FutureFillModel uses $\max(\text{stop_price}, \text{bar.Close})$ for buy stops, rather than the more realistic $\max(\text{stop_price}, \text{bar.Open})$. Users can override this with a custom FillModel, but the default is what most people run.

4.2 TradingView (Pine Script)

TradingView has the most sophisticated default approach. Its broker emulator infers the intra-bar price path by looking at where the Open sits relative to the High and Low. If the Open is closer to the High, it assumes price moved $\text{Open} \rightarrow \text{High} \rightarrow \text{Low} \rightarrow \text{Close}$. If the Open is closer to the Low, it assumes $\text{Open} \rightarrow \text{Low} \rightarrow \text{High} \rightarrow \text{Close}$. This determines which order gets filled first on ambiguous bars. Premium users can enable "Bar Magnifier" mode, which pulls actual lower-timeframe data instead of guessing. This is clever, but the heuristic has not been validated against tick data in any public documentation.

4.3 Backtrader

Backtrader, a popular open-source Python framework, does not support One-Cancels-Other (OCO) orders natively. Stop loss and take profit are separate orders, and if both trigger on the same bar, both execute using FIFO order processing with no path inference. The community has documented this extensively, with workarounds including lower-timeframe data feeds, the cheat-on-close parameter, or manual order management. The platform makes no assumption about which

order hits first. It just fills both. This is arguably the most honest approach, even if it produces the least useful results.

4.4 NinjaTrader

NinjaTrader's Standard fill resolution breaks each historical bar into three virtual sub-bars to approximate intra-bar movement, similar to TradingView's approach. Forum discussions indicate that when both a stop loss and entry fall on the same bar, the platform defaults to recording a loss. NinjaTrader also offers High Order Fill Resolution, where users can bring in a secondary data series (like 1-tick bars) for more accurate fills, and Market Replay for tick-by-tick historical playback. The advanced modes largely solve the problem but require extra data and setup.

4.5 MetaTrader 5

MetaTrader 5 is the most granular by default. Its strategy tester offers three modes: "Every Tick" (tick-by-tick simulation using real or reconstructed tick data), "1 Minute OHLC" (uses OHLC of one-minute bars), and "Open Prices Only" (fastest, least accurate). Every Tick mode, with high-quality data achieving 99.9% modeling quality, effectively eliminates the ambiguity problem. But many users select 1 Minute OHLC for speed, which has the same issue as every other platform.

4.6 Summary

Platform	Default Behavior	Ambiguity Handling	Fix Available
QuantConnect	OHLC bars; stops use bar.Close	Fills both SL and TP	Custom FillModel API
TradingView	OHLC with path heuristic	Infers $O > H > L > C$ or $O > L > H > C$	Bar Magnifier (Premium)
Backtrader	OHLC bars; FIFO execution	Fills both, no inference	Lower-TF data, cheat-on-close
NinjaTrader	3 virtual sub-bars per bar	Defaults to loss on same bar	High Fill Resolution, 1-tick bar
MetaTrader 5	Every Tick / 1min OHLC / Open	Tick mode solves it	Built-in tick simulation

Table 1: How each platform resolves intra-bar ambiguity.

5. Methodology

5.1 Data

I use E-mini NASDAQ-100 (NQ) futures continuous contract data at one-minute resolution from January 1, 2020 through November 17, 2025. The data comes from QuantConnect's historical feed using BackwardsRatio normalization for contract rolls. Extended market hours are included, covering the full 23-hour session. After filtering zero-range and zero-price bars, the dataset contains 2,064,460 bars.

5.2 Measuring Ambiguity

For each bar, I compute the range $R = H - L$. For each stop/target pair (S, T) in the test grid, a bar counts as ambiguous if $R \geq \max(S, T)$. I test 19 configurations from tight scalps (5/10 points) to wide swings (50/100 points). I break down ambiguity by hour of day and calendar year.

5.3 Performance Divergence

To quantify the impact, I compute the maximum theoretical gap between an optimistic resolution (all ambiguous bars are wins) and a pessimistic resolution (all ambiguous bars are losses). For N trades with ambiguity fraction f and stop/target sizes S and T , the max PnL divergence is $N \times f \times (S + T)$ points. This is the "ambiguity band," the range of possible outcomes that OHLC data simply cannot distinguish between.

6. Results

6.1 Bar Range Statistics

The mean one-minute bar range on NQ over the study period is 6.60 points, with a median of 4.50 and standard deviation of 7.22. The distribution is heavily right-skewed: the 90th percentile is 13.75 points, the 99th percentile is 33.50, and the max is 601.50. About 46.5% of all one-minute

bars have a range of 5 points or more, and 18.5% exceed 10 points. Figure 1 shows this distribution, with the ambiguous zone for a 10/10 scalp highlighted.

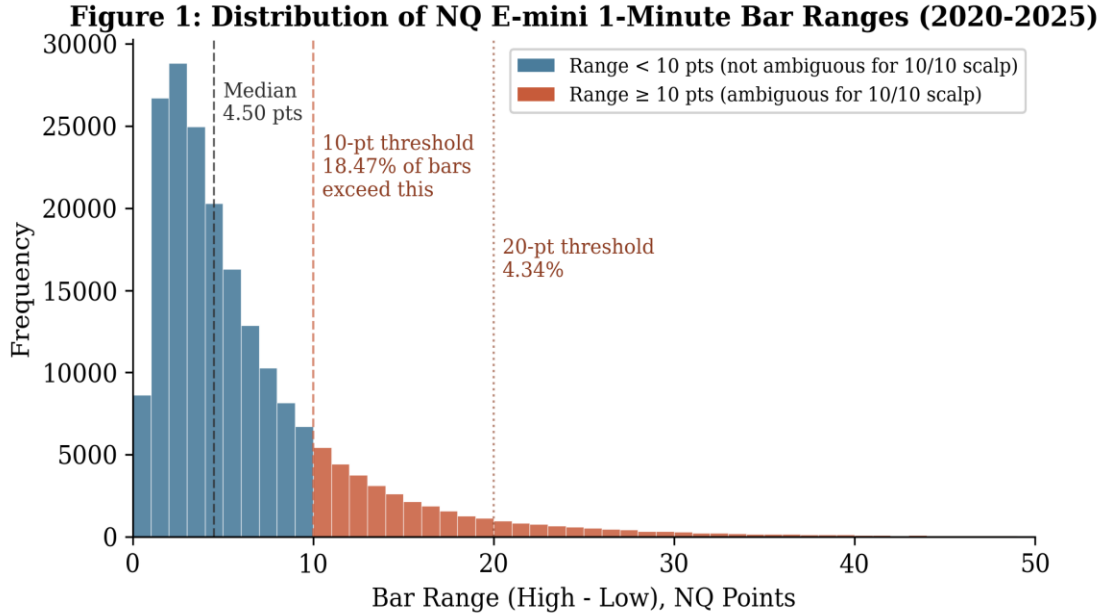


Figure 1: Distribution of NQ E-mini 1-minute bar ranges (2020-2025). Bars in red exceed the 10-point threshold, making them ambiguous for a symmetric 10/10 scalping setup.

6.2 Ambiguity Frequency

Stop (pts)	Target (pts)	Ambiguous Bars	% of Total	PnL Swing / 1k trades
5	10	381,365	18.47%	2,771 pts
5	15	176,729	8.56%	1,712 pts
5	20	89,617	4.34%	1,085 pts
10	10	381,365	18.47%	3,695 pts
10	20	89,617	4.34%	1,302 pts
10	30	28,785	1.39%	558 pts
15	15	176,729	8.56%	2,568 pts
20	20	89,617	4.34%	1,736 pts
30	30	28,785	1.39%	837 pts
40	40	12,222	0.59%	474 pts
50	50	6,483	0.31%	314 pts
50	100	845	0.04%	61 pts

Table 2: Ambiguity frequency and max performance divergence by stop/target pair. NQ E-mini, 1-min bars, Jan 2020 to Nov 2025 (N = 2,064,460). PnL Swing = max difference between all-win vs all-loss resolution per 1,000 trades.

The relationship between stop/target width and ambiguity is sharply nonlinear. For the tightest setup (5-point stop, 10-point target), 18.47% of bars are ambiguous, nearly one in five. A 20/20 setup drops to 4.34%. At 50/100, ambiguity is basically zero at 0.04%. The max divergence for a symmetric 10/10 scalp is 3,695 NQ points per 1,000 trades. At \$20 per point, that is roughly \$73,900 in phantom PnL that the platform is making up. Figure 2 shows what this looks like as a divergent equity curve.

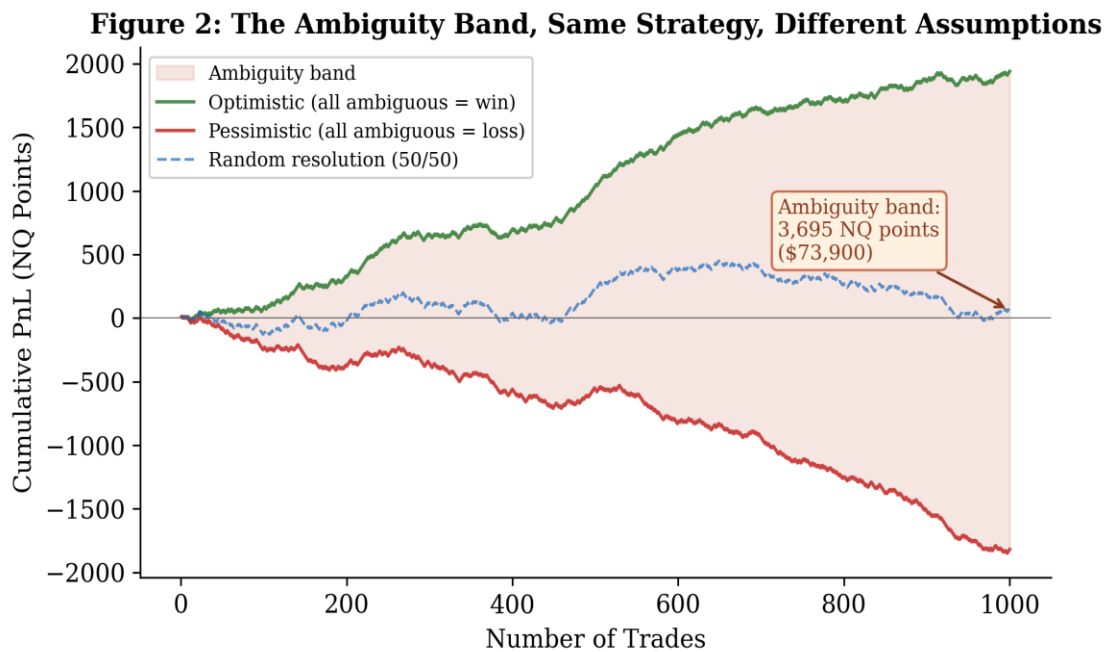


Figure 2: The ambiguity band for a 10/10 NQ scalp over 1,000 trades. The green line assumes every ambiguous bar is a win; the red line assumes every ambiguous bar is a loss. The shaded region is the range of outcomes OHLC data cannot distinguish between.

6.3 Time-of-Day Effects

Ambiguity is not spread evenly across the day. Using bars with 20+ point ranges as a proxy, the 9:00 AM and 10:00 AM hours (the first 90 minutes of regular trading) show rates of 18.24% and 18.82%, roughly 25 times higher than overnight hours which average below 1%. The 3:00 PM hour shows 10.69%. This matters because scalping strategies are disproportionately designed to trade the open, which is exactly when ambiguity is worst.

6.4 Volatility Regime Effects

Year	Bars	Mean	Median	P90	P95	≥20pt %
2020	350,842	6.21	4.25	12.75	17.25	3.55%
2021	353,378	5.14	3.50	10.75	14.50	2.14%
2022	353,266	8.11	5.75	16.50	21.50	6.25%
2023	351,455	5.13	3.50	10.75	14.25	1.95%
2024	346,563	6.53	4.50	13.50	18.25	4.04%
2025	308,956	8.77	6.00	18.50	25.25	8.64%

Table 3: NQ one-minute bar range statistics by year. Bars with 20+ point range vary 4.4x between the calmest year (2023: 1.95%) and the most volatile (2025: 8.64%).

Ambiguity gets worse in volatile markets. 2022, with aggressive Fed rate hikes, had 6.25% of bars above 20 points, three times the rate of 2023 (1.95%). 2025 is the worst in the dataset at 8.64%. This means backtest results for scalping strategies are least reliable exactly when volatility is highest, which is precisely when a strategy's risk management matters most.

7. Practical Guidelines

7.1 The Bar Range Ratio

I propose a simple rule of thumb for checking whether an OHLC backtest is trustworthy. The **Bar Range Ratio (BRR)** is the strategy's minimum exit distance (whichever is smaller, the stop or the target) divided by the median bar range for the instrument and timeframe. For NQ one-minute bars, the median range is 4.50 points. A strategy with a 10-point stop has a BRR of $10/4.50 = 2.22$. **BRR below 2:** Do not trust the backtest. Ambiguity exceeds 18%. You need tick data or sub-minute resolution.

BRR between 2 and 5: Treat results as a range, not a point estimate. Ambiguity is between 2% and 18%. Report both optimistic and pessimistic outcomes.

BRR above 5: Backtest is generally reliable. Ambiguity is below 2%, small relative to other unknowns like slippage and regime changes.

7.2 Working Without Tick Data

When tick data is not available, three approaches help manage ambiguity. First, report the ambiguity band alongside performance: the spread between best-case and worst-case resolution. Second, use conservative resolution, assume the stop is hit first on every ambiguous bar. If the strategy is still profitable under that assumption, it is robust. Third, run Monte Carlo simulations over ambiguous bars, randomly assigning wins and losses to build a distribution of outcomes instead of a single equity curve.

7.3 Choosing Resolution

If you can choose your data resolution, use the finest available up to the point where ambiguity drops below an acceptable level for your stop/target width. For NQ scalps with 10 to 20 point exits, one-second or tick data is strongly recommended. One-minute bars are not good enough for these strategies, as the 18.47% ambiguity rate shows.

8. Discussion

8.1 What This Means for Retail Traders

The implications for the retail algorithmic trading community are significant. Strategies shared on forums, blogs, and social media are almost always backtested on OHLC data at one-minute resolution or coarser. Many are scalping systems with tight stops, exactly the type most affected by ambiguity. A meaningful fraction of published "profitable" scalping strategies likely have overstated performance, not because anyone is lying, but because the testing infrastructure cannot accurately resolve trade outcomes for their strategy type.

8.2 What Platforms Should Do

Platforms should implement three things. First, flag trades where both the stop and target fall within the current bar's range. Second, provide an ambiguity report showing what fraction of trades

were resolved by assumption rather than data. Third, show the ambiguity band as a standard output alongside the equity curve. TradingView's Bar Magnifier and MetaTrader 5's Every Tick mode are steps in the right direction, but they are opt-in features that most users never enable.

8.3 Limitations

This study focuses on a single instrument (NQ futures). Different instruments with different volatility, tick sizes, and session structures will have different ambiguity distributions. The divergence estimates are theoretical maximums; some platform heuristics may get it right on a portion of ambiguous bars. Without tick data validation, I cannot say how often. I only analyze one-minute bars; extending to 5-minute and 15-minute bars would give a more complete picture. The BRR thresholds are NQ-specific and may need adjustment for other instruments.

8.4 Future Work

Several extensions would strengthen this work. Validating platform heuristics against tick data would establish which approach is most accurate. Extending across multiple instruments (ES, CL, GC, BTC futures) would generalize the BRR framework. Running concrete strategies through all three assumptions (optimistic, pessimistic, random) would produce divergent equity curves as a clear illustration. Surveying publicly shared strategies to estimate how many are vulnerable to this problem would quantify the scope of the issue in the community.

9. Conclusion

This paper identifies and quantifies the intra-bar ambiguity problem in OHLC-based backtesting. Using over 2 million one-minute bars of NQ futures data spanning six years, I show that for typical scalping setups, nearly one in five bars is ambiguous, meaning the backtest engine cannot determine the trade outcome and has to guess. The resulting uncertainty is large: up to \$73,900 per 1,000 trades for a symmetric 10-point scalp. Five major platforms each handle this differently,

producing different results for identical strategies on identical data. I propose the Bar Range Ratio as a practical tool for assessing backtest reliability. The bottom line is straightforward: if your stop or target is smaller than twice the median bar range, your OHLC backtest is unreliable, and no amount of statistical rigor in strategy selection can fix inaccuracy in the simulation itself.

References

- Almgren, R., & Chriss, N. (2001). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2), 5-39.
- Bailey, D. H., Borwein, J. M., López de Prado, M., & Zhu, Q. J. (2014). Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the AMS*, 61(5), 458-471.
- Cont, R., Stoikov, S., & Talreja, R. (2010). A stochastic model for order book dynamics. *Operations Research*, 58(3), 549-563.
- Harvey, C. R., & Liu, Y. (2015). Backtesting. *Journal of Portfolio Management*, 42(1), 13-28.
- White, H. (2000). A reality check for data snooping. *Econometrica*, 68(5), 1097-1126.
- QuantConnect. (2024). LEAN Engine documentation: Trade fills. quantconnect.com/docs/v2/writing-algorithms/reality-modeling/trade-fills/key-concepts
- TradingView. (2024). Pine Script documentation: Strategies. tradingview.com/pine-script-docs/concepts/strategies/
- NinjaTrader. (2024). Understanding historical fill processing. ninjatrader.com/support/helpguides/nt8/understanding_historical_fill_.htm
- MetaQuotes. (2024). MetaTrader 5 strategy tester. metatrader5.com/en/automated-trading/strategy-tester

Appendix A: Corrected Fill Model for QuantConnect

This Python code implements a corrected fill model for QuantConnect futures backtesting. It uses `bar.Open` instead of `bar.Close` for stop market fills:

```
class RealisticFutureFillModel(FillModel):
    """
    Buy stop fill = max(bar.Open, stop_price)
    Sell stop fill = min(bar.Open, stop_price)
    """
    def StopMarketFill(self, asset, order):
        bar = asset.GetLastData()
        if bar is None:
            return super().StopMarketFill(asset, order)
        stop_price = order.StopPrice
        if order.Direction == OrderDirection.Buy:
            if bar.High < stop_price:
                return super().StopMarketFill(asset, order)
            fill_price = max(bar.Open, stop_price)
        else:
            if bar.Low > stop_price:
                return super().StopMarketFill(asset, order)
            fill_price = min(bar.Open, stop_price)
        fill = OrderEvent(order, bar.EndTime, OrderFee.Zero)
        fill.FillPrice = fill_price
        fill.FillQuantity = order.Quantity
        fill.Status = OrderStatus.Filled
        return fill
```

Usage: `self.Securities[symbol].SetFillModel(RealisticFutureFillModel())`